

Sentinel — An LLM-Powered Agentic Data Pipeline Observability System

A portfolio project build guide for a Data Science master's student.

1. One-line summary

Sentinel is an **agentic observability system** that sits alongside a running data pipeline. It continuously watches the data, detects anomalies, uses an LLM to diagnose the likely root cause and propose a fix, and — for a small set of safe, reversible actions — **asks a human for approval before acting**. Every incident, decision, and outcome is remembered, so the system gets better over time. It runs the full **Observe → Reason → Propose → Approve → Act → Remember** loop, with a human in the loop at the gate.

2. Why this project (market context)

This sits in the **data observability** space — a real, funded, growing category (Monte Carlo, Atlan, Anomalo, Soda). The agentic evolution (observe → reason → act → learn) is exactly where the field is heading in 2026, and the safe rollout pattern the industry recommends is *suggest-first, act only with approval*. You are **not** trying to beat the incumbents; you are building a focused, well-architected slice of one and proving you understand where the field is going.

The project is organized around a recognized **six-layer reference architecture for agentic data systems**. The power move for your portfolio is to present the *full* reference architecture and then be explicit about *which layers you implemented at full fidelity, which are intentionally lightweight, and which are deliberately constrained* — and why. That contrast is the maturity signal.

Not an AI code-assistant (Cursor / Copilot). Those help you write code in an editor. Sentinel watches *running data* and reacts to *live data problems*. State this in your README — it shows you understand the landscape.

The three skills it showcases, in one project: **data engineering** (a real orchestrated pipeline), **anomaly detection** (statistics / ML on run-level metrics), and **applied LLM / agent design** (root-cause reasoning, memory-augmented retrieval, governed action, and rigorous evaluation of all of it).

3. Problem statement

Data pipelines fail silently. A schema change upstream, a dropped batch, a spike in null values, or a distribution shift can corrupt every downstream dashboard and model — often undetected for hours or days. Traditional monitoring catches *infrastructure* failures (a job crashed) but not *data* failures (the job ran fine but produced bad data). Hand-writing data-quality rules for every column does not scale, and purely reactive alerting still leaves a human to diagnose and fix everything from scratch, every time.

Sentinel learns the normal behaviour of each pipeline run and watches two signal streams together — the *data* itself and the pipeline's *operational state* (job status, duration, retries). It detects statistical anomalies without hand-written rules and uses an LLM — informed by the pipeline's stated intent, its memory of past incidents, and the operational context — to **correlate symptoms with causes** and produce an actionable incident report: root cause, severity, and a suggested remediation. For a constrained set of safe, reversible actions, it can carry out the fix **once a human approves**, then records the outcome to improve future responses.

What Sentinel covers

Sentinel targets **silent data-quality failures** — the ones that pass infrastructure monitoring because the job exits cleanly but the data is wrong — and correlates them with **operational signals** to explain *why* they happened. It complements crash-monitoring tools (Airflow alerts, Datadog, PagerDuty); it does not replace them.

Failure	Type	Detected by
Late / missing data	Data	Freshness
Too few / too many rows	Data	Volume baseline
Schema drift (column added/removed/retyped)	Data	Schema-hash
Null / completeness spike	Data	Null-rate baseline
Distribution / value drift	Data	PSI / KS test
Job failed / skipped / slow / retried	Operational	Orchestrator state (context for correlation)
Pure infra crash (OOM, network, auth)	Operational	Surfaced as context; primary alerting left to existing tools

The differentiator is the **correlation**: connecting a *data* symptom (" `amount` nulls spiked in the transactions feed") to its *operational* cause ("upstream job `enrich_balances` failed and was skipped").

4. The architecture: six layers

This is the conceptual spine of the project. Each layer below lists **what it does** and **the fidelity you should build it at**.

(1) Intent Layer — *light, worth adding*

Declares the *purpose* of each pipeline, not just its steps: business goal, data consumers, and expectations for freshness, accuracy, and reliability (SLAs). This lets the system prioritize dynamically — a freshness breach on a dataset feeding a live dashboard is more urgent than on an archival table. **Build:** a small per-dataset YAML config. It feeds severity and prioritization in the Reasoning layer. Cheap to build, high signal.

(2) Observability Layer — *full, core*

Continuous visibility into pipeline health and data quality, across **two streams**: *data signals* (freshness, volume, schema drift, null rates, distribution drift, SLA breaches) and *operational signals* read from the orchestrator (job status, run duration, retries, last-run time, exit status). Together they let the system see not just *that* the data is wrong but *what the pipeline was doing* when it went wrong. **Build:** metadata store + data-metrics computation + a thin operational-signals collector + the anomaly-detection engine. This is the bedrock — build it well.

(3) Reasoning Engine — *full, core*

The decision-making core. Interprets anomaly signals, performs root-cause analysis, proposes a fix, and assigns severity — *in context*, using the pipeline's intent, **retrieved similar past incidents from Memory**, and the **operational signals** so it can *correlate a data symptom with its pipeline-side cause*. This correlation — "the null spike was caused by that skipped upstream job" — is what makes it intelligent rather than a generic alert. **Build:** an LLM (Claude API) call with carefully assembled context, returning a structured JSON incident report.

(4) Action Layer — *deliberately constrained*

Executes decisions by interacting with orchestration and infrastructure (re-run a job, quarantine a bad batch). **Critical design choice:** keep this to a small set of **safe, reversible** actions, every one **gated behind human approval** via the Governance layer. No autonomous, irreversible changes to production data or schema — those are explicitly out of scope. **Build:** an action registry with 2 actions to start; the executor only runs after approval.

(5) Memory Layer — *full, your differentiator*

Stores past incidents, the decisions made, and their outcomes. Before reasoning about a new anomaly, the system **retrieves similar past incidents** to inform the diagnosis (RAG over incident history). Over time it learns which fixes actually worked for which kinds of anomaly.

Build this properly — it turns a stateless "LLM wrapper" into a system that *demonstrably improves*, and it gives you a headline evaluation result (with-memory vs. without-memory).

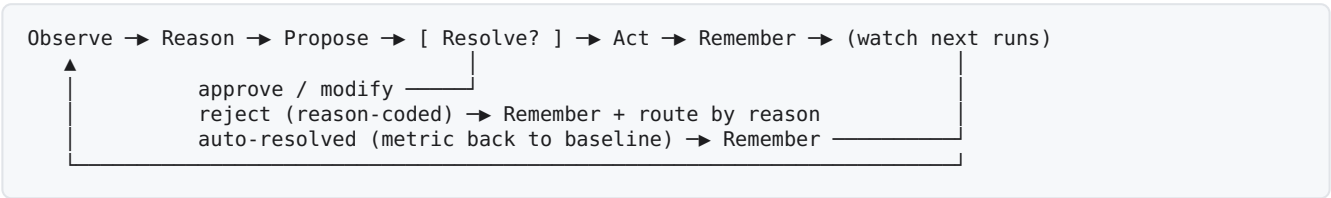
(6) Governance Layer — *light-to-medium, worth adding*

Enforces policy: which actions can be auto-approved, which need human sign-off, which are blocked. Logs every proposed action, decision, and outcome for full traceability. This is what makes "ask before acting" a principled design rather than just a limitation. **Build:** a risk-tier policy config + an approval gate + an append-only audit log.

Fidelity at a glance

Layer	Build fidelity	Where in the repo
1. Intent	Light config	<code>intent/</code>
2. Observability	Full (core)	<code>observability/</code>
3. Reasoning	Full (core)	<code>reasoning/</code>
4. Action	Constrained (2 safe actions, gated)	<code>action/</code>
5. Memory	Full (differentiator)	<code>memory/</code>
6. Governance	Light-medium	<code>governance/</code>

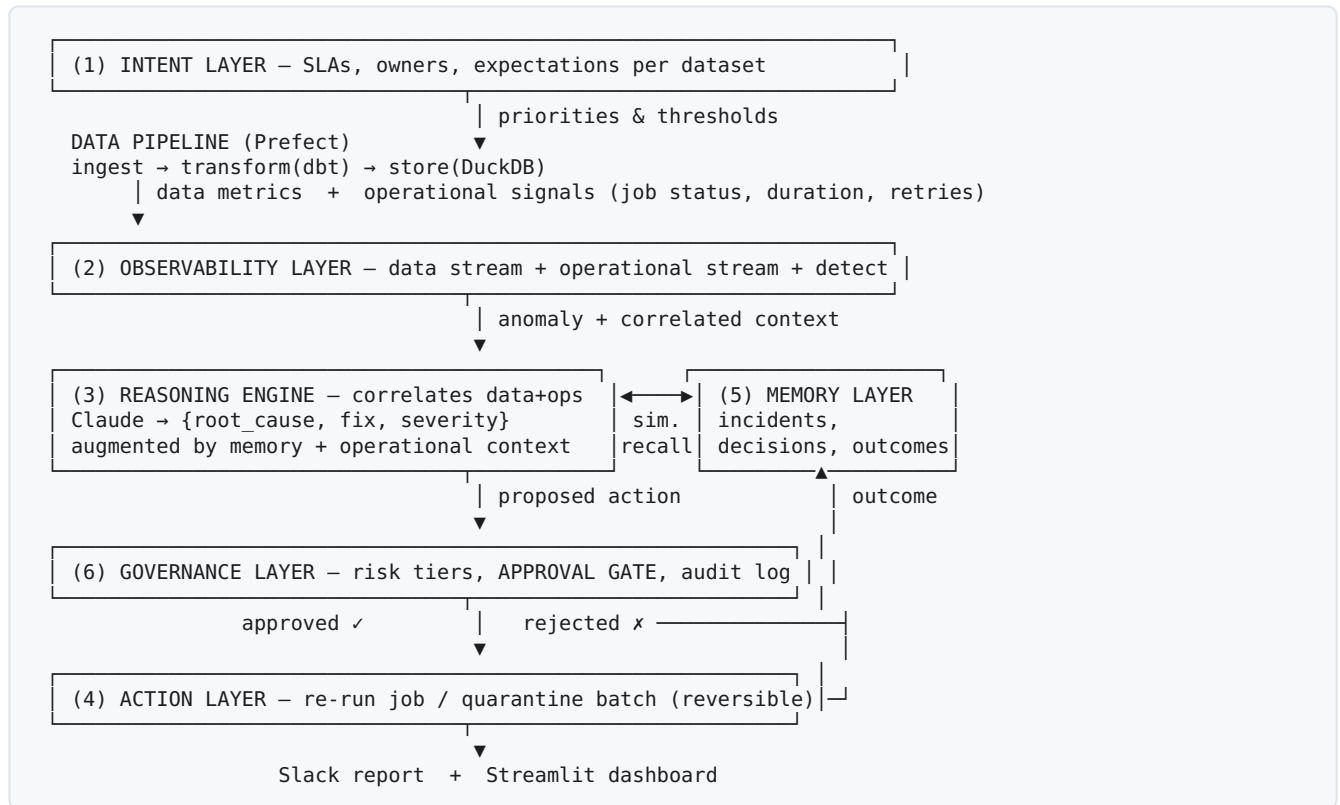
5. The agentic loop



- **Observe** — pull run-level signals from the Observability layer, with a one-run debounce on low-severity anomalies so transient blips never escalate.
- **Reason** — diagnose root cause using intent + retrieved memory; classify transient vs. systemic; propose a fix + severity.
- **Propose** — present the incident report and the suggested action.
- **Resolve** — a human approves, modifies, or rejects *with a reason* (see Section 7). Governance gates by risk tier.
- **Act** — only on approval, execute the reversible action.
- **Remember** — store the incident, the resolution (including the rejection reason), and the outcome.
- **Watch** — keep monitoring subsequent runs; if the metric returns to baseline, auto-mark the incident resolved and record time-to-resolution — even if a human fixed it outside Sentinel.

Every path — approve, modify, reject-with-reason, and auto-resolve — writes back to Memory.
Rejections are signal, not noise.

6. System architecture diagram



The **fault-injection harness** plugs into ingest to deliberately corrupt data with known labels — your evaluation ground truth.

7. Risk-tiered action & approval design

Not all actions carry equal risk, so the approval gate is **not** a single yes/no. Classify each action and let Governance decide the gate:

Action	Risk tier	Reversible?	Gate
Re-run a failed/late job	Safe	Yes	One-click approve
Quarantine a bad batch (move to quarantine table)	Safe	Yes (un-quarantine)	One-click approve
Backfill a window from source	Medium	Mostly	Typed confirmation + preview
Patch / modify data in place	Risky	No	Blocked (manual only — out of scope)
Alter schema	Risky	No	Blocked

Design rules: - **Preview before acting.** Always show what the action will do (a diff / dry-run / predicted effect) before approval. Never approve a black box. - **Prefer reversible actions** and keep an undo path (quarantine ↔ un-quarantine). - **Sandbox the executor.** Actions touch your orchestrator (Prefect re-trigger) and warehouse (quarantine table) only. Design it *as if* it were production. - **Audit everything.** Append-only log of: proposed action, decision, who/when, and outcome. - **Close the loop into Memory.** The outcome ("did the fix resolve the anomaly?") is written back so future suggestions improve.

Resolution handling — "reject" is not one thing

A plain reject button hides at least three different situations and throws away your best learning signal. Make rejection **reason-coded**, and route each reason:

Resolution	What it means	What the system does
Approve / Modify	Fix is right (or right after editing)	Execute after the gate; record outcome
Reject — <i>not a real problem</i>	False positive / ignorable	Label as FP; create a suppression rule or loosen the Intent expectation so similar anomalies stop firing
Reject — <i>I'll fix manually</i>	Real, handled by a human	Mark <i>acknowledged-manual</i> ; capture what they did → store in Memory for next time
Reject — <i>wrong diagnosis/fix</i>	Anomaly real, proposal wrong	Keep incident open; log the proposal as a negative signal for Reasoning/retrieval
Snooze	Real, not now	Defer and re-surface later

Two feedback loops fall out of this: - **Suppression loop (false positives):** labelled FPs feed back into thresholds and Intent expectations, so your false-positive rate *falls over time*. Track it — a declining FP rate is proof the system learns. - **Knowledge-capture loop (manual fixes):** captured human resolutions become Memory, so the next similar incident surfaces "last time a human resolved this by doing X."

Plus a noise guard at the front: **debounce low-severity anomalies by one run** — a blip that self-corrects next run never pages anyone; only persistent or high-severity anomalies escalate immediately.

Start with just **two actions** (re-run, quarantine). The *mechanism* — risk tiers + reason-coded resolution + suppression + audit + outcome tracking — is what's impressive, not the breadth.

8. Objectives and scope

In scope: 1. A real, scheduled data pipeline ingesting/transforming a public dataset. 2. Intent configs (SLAs/expectations) per dataset. 3. Observability across two streams: data metrics + operational signals (job status, duration, retries), plus a rules-plus-statistics anomaly engine. 4. A synthetic fault-injection harness for labelled ground truth. 5. A Memory layer with incident storage and similarity retrieval. 6. An LLM Reasoning engine producing structured, memory-augmented incident reports. 7. A constrained Action layer (2 reversible actions) behind a risk-tiered, reason-coded approval gate (approve / modify / reject-with-reason / snooze). 8. Governance: risk policy + suppression rules + audit log + outcome tracking, with auto-resolution detection from subsequent runs and a one-run debounce on low-severity anomalies. 9. Routing (Slack) and a Streamlit dashboard with approve/reject/modify controls. 10. Rigorous evaluation of detection, report quality, the memory effect, and action correctness.

Out of scope (say so explicitly): - Autonomous, unattended action — every action requires approval. - Irreversible production changes (in-place data edits, schema alters). - Multi-tenant / enterprise scale; full cross-system lineage graphs.

9. What makes it portfolio-worthy

- **Measured detection** — precision / recall / F1 against injected faults, plus detection latency.
 - **The memory result** — a with-memory vs. without-memory comparison showing retrieval improves diagnosis quality. Few student projects have this.
 - **Cross-signal correlation** — connecting a data symptom to its operational root cause (e.g., a null spike caused by a skipped upstream job). The "real RCA" capability commodity tools don't surface.
 - **Governed autonomy** — risk-tiered, reason-coded approval gate with a full audit trail; a sophisticated, interview-ready design point.
 - **A system that learns from feedback** — false-positive rate *falling over time* as suppression rules accumulate. A rare, convincing "it actually improves" result.
 - **Evaluated LLM output** — rubric-scored reports, not a single cherry-picked example.
 - **A clean architecture** mapped to the six layers, an honest "implemented subset vs. reference" framing, a demo GIF, and a write-up explaining trade-offs.
-

10. Recommended tech stack

Layer / concern	Recommended	Notes / alternatives
Orchestration	Prefect	Lighter than Airflow for solo work. Alt: Airflow (more "industry standard" on a CV), Dagster.
Transformation	dbt	Industry-standard; optional if short on time.
Storage / warehouse	DuckDB	Zero-setup, local, fast. Alt: Postgres, BigQuery free tier.
Data-quality checks	Custom Python (+ optional Soda Core / Great Expectations)	Custom shows understanding; library shows tool familiarity.
Anomaly detection	scipy / statsmodels , optional Prophet	Rolling z-score / IQR; PSI or KS test for drift; seasonal model for volume.
Reasoning LLM	Anthropic Claude API (<code>claude-sonnet-4-6</code>)	JSON-only structured output.
Memory store + retrieval	Embeddings + a vector store (Chroma / FAISS / <code>pgvector</code>)	Embed incident summaries; retrieve top-k similar. SQLite/Postgres for the records.
Metadata / audit store	SQLite or Postgres	Run history + append-only audit log.
Dashboard	Streamlit	Approve / Reject / Modify UI.
Routing	Slack incoming webhook / SMTP	"Reports to the department."
Tooling	git, <code>uv</code> / <code>poetry</code> , <code>ruff</code> , <code>pytest</code>	Repo hygiene is itself a signal.

Make a conscious call on Prefect + dbt. They're valuable CV keywords, but orchestration is *incidental* to the interesting part (the agentic loop). If time gets tight, a plain Python scheduler frees you to spend effort on detection + reasoning + memory — the parts that actually differentiate the project. Choose deliberately rather than defaulting in.

11. Dataset choice

Primary (recommended): PaySim — synthetic mobile-money transactions. A large, time-stepped, *synthetic* transaction log (no privacy concerns) with interpretable, named columns — `step` (hourly time index), `type` , `amount` , origin/destination accounts, and pre/post balances, plus fraud labels. It's an ideal fit for Sentinel because:

- **Named, interpretable columns** make the LLM's root-cause reports read naturally ("`amount` distribution shifted", "`oldbalance0rg` is suddenly null") — unlike anonymized fraud datasets whose `V1...V28` columns produce meaningless narratives.

- **A real time axis** (`step`) lets you slice the data into scheduled "daily" batches, so freshness and volume anomalies are genuinely meaningful.
- **High volume** (millions of rows) is enough to bootstrap an incident corpus for the Memory evaluation.
- **A high-stakes, relatable domain.** Framing it as "*a daily transactions pipeline feeding a fraud-scoring model and a regulatory report*" gives the Intent layer real teeth — concrete freshness/completeness SLAs and compelling severity reasoning.

Suggested pipeline to monitor (so there's something realistic to observe):

`raw transactions` → `cleaned/typed` → `enriched (balances, account joins)` → `daily fraud-scoring features`. Each stage is a job whose operational signals you collect and whose output data you profile.

Strong alternatives: - **Lending Club loan data** — real, ~145 columns with an issue-date field; richer schema (great for schema-drift and many-column null storytelling), at the cost of more messiness. Batch by issue month. - **IBM Transactions for Anti-Money-Laundering** — synthetic, timestamped, with laundering labels; excellent if you want an explicit AML angle.

Avoid for this project: the PCA-anonymized credit-card fraud dataset (`V1...V28`). Great for fraud ML, poor for observability — the anonymized columns make root-cause narratives meaningless.

Your **fault-injection harness** sits on top of whichever you pick, supplying the labelled anomalies — including the "upstream job failure *causes* a data fault" scenarios — that power evaluation.

12. Methodology (phase by phase)

Phase 0 — Setup & scoping (½ day). Repo, env, README skeleton with problem statement + architecture diagram. Anthropic API key.

Phase 1 — Pipeline (2-3 days). Prefect flow: ingest a batch → (optional dbt) transform → load to DuckDB. Schedule it; confirm clean end-to-end.

Phase 2 — Intent layer (½ day). Per-dataset YAML: owner, consumers, freshness SLA, accepted null %, key columns. Loaded by later layers.

Phase 3 — Observability: metadata & metrics (1-2 days). Per run, log row count, freshness, per-column null rate, schema hash, and summary stats for key numeric columns. Also capture **operational signals** from the orchestrator — job status, run duration, retries, last-run time, exit status — as a second stream.

Phase 4 — Observability: detection engine (2-3 days). Compare each metric to its rolling baseline (z-score/IQR), schema-hash mismatch for drift, freshness threshold for staleness, PSI/KS for distribution drift. Emit structured anomaly objects. Add a one-run debounce so low-severity blips that self-correct don't escalate.

Phase 5 — Fault-injection harness (1-2 days). Inject, with known labels: row drops, column nulling, distribution shift, schema change, stale data. **Your evaluation engine — don't skip.**

Phase 6 — Memory layer (2 days). Incident store + embeddings + top-k similarity retrieval. Build before/with Reasoning so reasoning can consume it.

Phase 7 — Reasoning engine (2-3 days). Assemble context (anomaly + recent history + schema + intent + git SHA + retrieved similar incidents + **operational signals**). Prompt Claude to *correlate the data symptom with operational state* and return JSON only: `{severity, likely_root_cause, caused_by, evidence, suggested_action, confidence}`. Parse defensively; store the incident.

Phase 8 — Action, Governance & resolution (3-4 days). Action registry (re-run, quarantine), risk-tier policy, reason-coded approval gate (approve / modify / reject-with-reason / snooze), suppression rules, preview/dry-run, executor (post-approval only), append-only audit log, auto-resolution detection from subsequent runs, and the outcome write-back into Memory.

Phase 9 — Routing & dashboard (1-2 days). Slack delivery for high-severity incidents; Streamlit dashboard: health timeline, metric charts with anomaly markers, incident feed, and Approve/Reject/Modify controls with action preview.

Phase 10 — Evaluation (3 days). First bootstrap an incident history: replay many injected faults so Memory holds a real corpus — otherwise retrieval returns nothing and the memory result won't appear. Then run many clean + faulty batches: - **Detection:** precision / recall / F1 vs. injected faults; detection latency; rules-only vs. rules+stats ablation. - **Report quality:** rubric (root cause correct? action appropriate? severity reasonable?) on a sample; optional LLM-as-judge. - **Root-cause attribution:** inject data faults *caused by* an upstream job failure and measure whether the reasoner correctly blames the operational cause rather than the data. - **Memory effect:** with-memory vs. without-memory diagnosis quality — your headline result. - **Action correctness:** of approved actions, how many actually resolved the anomaly; false-positive rate on clean runs (report honestly). - **Learning curve:** feed reject-as-false-positive labels over successive runs and plot the false-positive rate falling.

Phase 11 — Polish & packaging (2-3 days). Architecture diagram, demo GIF, thorough README, write-up/blog post on design trade-offs and results.

13. Project folder structure

```
sentinel/
├── README.md
├── pyproject.toml
├── .env.example          # ANTHROPIC_API_KEY, SLACK_WEBHOOK_URL
├── docs/
│   ├── architecture.png
│   └── writeup.md        # design doc / blog post
├── data/                 # DuckDB file, raw batches (gitignored)
├── intent/               # (1) Intent layer
│   └── datasets/*.yaml  # SLAs, owners, expectations
├── pipeline/
│   ├── flows.py          # Prefect flows
│   ├── ingest.py
│   ├── transform/       # dbt project (or transform.py)
│   └── faults.py         # fault-injection harness
├── observability/       # (2) Observability layer
│   ├── store.py          # metadata store read/write
│   ├── metrics.py        # data: freshness/volume/null/schema/dist
│   ├── operational.py    # ops: job status, duration, retries, exit
│   └── detection/
│       ├── engine.py
│       ├── rules.py
│       └── statistical.py # z-score, IQR, PSI/KS
├── memory/              # (5) Memory layer
│   ├── store.py          # incident/decision/outcome records
│   ├── embed.py         # embed incident summaries
│   └── retrieve.py       # top-k similar incidents
├── reasoning/           # (3) Reasoning engine
│   ├── context.py        # assemble context bundle (+ memory + intent)
│   ├── prompts.py        # JSON-only system + report prompts
│   └── reporter.py       # call Claude, parse, store incident
├── action/              # (4) Action layer
│   ├── registry.py       # available actions + risk tiers
│   ├── preview.py        # dry-run / predicted effect
│   └── executor.py       # runs ONLY after approval
├── governance/          # (6) Governance layer
│   ├── policy.py         # risk-tier → gate mapping
│   ├── approval.py       # reason-coded gate (approve/modify/reject/snooze)
│   ├── suppression.py    # FP suppression rules + Intent loosening
│   ├── resolution.py     # auto-resolution detection from later runs
│   └── audit.py          # append-only audit log + outcome → memory
├── routing/
│   └── slack.py
├── dashboard/
│   └── app.py            # Streamlit, with approve/reject/modify
├── evaluation/
│   ├── run_experiments.py
│   ├── detection_metrics.py
│   ├── report_rubric.py
│   └── memory_ablation.py # with vs. without memory
├── tests/
│   └── test_*.py
```

14. Key implementation notes

- **JSON-only LLM output.** System prompt forbids preamble and markdown fences; strip stray fences and `json.loads` defensively; wrap every call in `try/except`.

- **Context, not raw data.** Send the LLM the anomaly, recent metric history, schema, intent, recent code/schema-change signal, and retrieved similar incidents — never the dataset itself. Context selection *is* the interesting engineering.
- **Memory retrieval drives the differentiator.** Embed a short structured summary of each incident; retrieve top-k by similarity; include them (with their outcomes) in the reasoning context.
- **Approval gate is risk-aware**, not binary — drive it from the action's risk tier (Section 7).
- **Lineage can be lightweight** — a small hand-written source→transform→output map for one pipeline is enough context.
- **Everything reversible and audited.** Prefer reversible actions; log every proposal, decision, and outcome.
- **Mind LLM cost & latency.** Don't fire a model call on every raw anomaly — call only on deduplicated, non-suppressed, above-threshold incidents, and cache/batch where you can. Keeps cost sane and signals production awareness.
- **Reproducibility = credibility.** Seed the fault injection, version every config, and document a couple of real failure cases ("here's where it breaks and why"). That reads as more mature than claimed perfection.

15. Evaluation summary (put this in the README)

Dimension	Metric	Aim for
Detection	Precision / Recall / F1 vs. injected faults	F1 $\geq$ 0.8 on clear faults
Detection	Mean detection latency	Within 1 run of fault
Report	Root-cause correctness (rubric)	$\geq$ 70% of high-severity cases
Correlation	Root-cause attribution (data symptom → infra cause)	Correct on a clear majority of linked faults
Memory effect	Diagnosis quality, with vs. without memory	A measurable lift — <i>headline result</i>
Action	% of approved actions that resolved the anomaly	Majority
Robustness	False-positive rate on clean runs	Low, reported honestly
Learning	False-positive rate over time (with suppression feedback)	A downward trend — proof it learns

An honest false-positive rate and a couple of documented failure cases beat a claim of perfection.

Define the rubric before you build, and protect this phase. The demo is the fun part; the metrics are what get you hired, and they're the part you'll be tempted to defer. Two prerequisites for the numbers to be real: (1) the fault-injection harness gives you

ground truth, and (2) Memory must be pre-populated with an incident corpus (see Phase 10) before the with-vs-without comparison means anything.

16. Stretch goals (only after the core works)

- **LLM-proposed checks:** have the model suggest new data-quality rules from observed history.
- **Multi-pipeline support** with a richer lineage graph.
- **Confidence-gated auto-approve** for the safest tier once memory shows a fix is consistently correct — still fully audited.
- **"Predicted effect" simulation:** run a proposed action on a copy and report the predicted result before approval.

17. Build order, milestones & timeline

Build it as **independent, demoable milestones**, not one monolithic push. Each milestone is presentable on its own, and "I started here, then extended it..." tells a better growth story in interviews than one giant deliverable. Finish each before starting the next.

Milestone	Delivers	Portfolio-ready?
M1 — Observe	Pipeline + Observability + Streamlit dashboard	Yes — a real project on its own
M2 — Reason	LLM root-cause + structured incident reports	Yes — clearly stronger
M3 — Remember	Memory layer + the with-vs-without-memory result	Yes — your differentiator
M4 — Act & Govern	Risk-tiered, reason-coded approval gate + suppression + auto-resolution	Yes — the flagship version

Stop at any milestone and you still have something worth showing. The two things that most separate this from the crowd — the **Memory effect** and the **governed resolution gate** — live in M3 and M4, so push to at least M3 if you can.

Time estimates

Pace	Duration	Best for
Focused full-time	~4-5 weeks	All four milestones (flagship)
Part-time alongside studies	~8-10 weeks	Realistic for a master's workload
Minimum viable	~1.5-2 weeks	M1 + M2 only; defer M3 / M4

Even the minimum version is a real project; the full layered version is genuinely distinctive.

18. How to talk about it in interviews

Lead with the *problem* ("pipelines fail silently; rule-writing doesn't scale"). Then the *judgement*: "I structured it around the six-layer agentic architecture, built Observability and Reasoning at full fidelity, made Memory my differentiator, and deliberately constrained Action behind a risk-tiered approval gate — because the industry doesn't yet trust autonomous remediation." Then the *evidence*: your F1, your with-vs-without-memory lift, your action-resolution rate, and your false-positive rate falling as feedback accumulates. Problem → judgement → evidence is the arc that lands.